# Coding Assessment: TaskFlow

**Deliverable:** A GitHub repository link + a Loom/screen-recording demo (4 to 6 minutes).

## Requirements

Build **TaskFlow**, a collaborative task-board app (a small Trello/Jira-style tool) where users create projects, invite other users as members, and manage tasks on a board, with live updates as the team works.

This has real relationships, real state, and real edge cases. Read all of it before you start coding, and plan your data model first.

### Core scope (must build)

**Authentication and accounts**

1. Sign up with name, email, password. Validate email format and enforce password rules you define and document.
2. Log in / log out using **JWT with a refresh-token flow**: a short-lived access token and a longer-lived refresh token. Document how you store each and why.
3. Passwords hashed with bcrypt or argon2. **Plaintext is an automatic fail.**
4. A user can only see and modify data for projects they are a member of. Enforce this on the backend. We will call your API directly to test it.
5. Unauthenticated requests to protected endpoints return 401. Expired access token returns 401 and the client transparently refreshes and retries.

**Projects and membership (many-to-many)** 6. A user can create projects. The creator is the project **owner**. 7. The owner can invite other registered users (by email) as **members**. A user belongs to many projects; a project has many members. Model this properly. 8. Roles within a project: **owner** (can invite/remove members, delete the project) and **member** (can manage tasks but not membership). Enforce on the backend. 9. Removing a member must not delete the tasks they created; it just revokes their access. Deleting a project removes its tasks and memberships cleanly (no orphaned references).

**Tasks and the board** 10. Within a project, create tasks with: title, description, status (`To Do`, `In Progress`, `Done`), priority (Low/Medium/High), optional due date, and an optional assignee (a project member). 11. List, edit, delete tasks. Move a task between statuses (the board columns). 12. A board view with columns by status; tasks appear in the right column. 13. Filter tasks by assignee and priority at the same time, and search by title. 14. **Server-side pagination AND sorting** for a project's task list/backlog view (by priority, due date, or date created). The frontend sends page, filters, search, and sort; the backend does the work. No loading-everything-then-slicing.

**Validation and derived data** 15. Validation: reject an empty title, reject a due date in the past on creation, reject assigning a task to someone who is not a member of that project. Clear error messages, no crashes. 16. When a task is moved to `Done`, record a "completed date". If it is moved back out of `Done`, clear it. 17. A task cannot be assigned to a user who has been removed from the project; handle this case (e.g., auto-unassign or block).

**Cross-user interaction (the hard part)** 18. When the owner assigns a task to a member, that member sees it in their personal "Assigned to me" view (across all their projects). 19. A member can comment on a task; comments show author and timestamp and are visible to all project members. 20. Only the task's

assignee or the project owner can mark a task `Done`. A non-assignee member trying to do so gets a clear error. Enforce on the backend.

**Activity log** 21. Record key events per project: task created, task moved, task assigned, member invited, member removed, comment added. Show a reverse-chronological activity feed on the project page.

**Real-time updates (WebSockets)** 22. Use WebSockets (Socket.io, native `ws`, or your framework's equivalent) for live updates. Polling does not satisfy this requirement; document your choice in the README. 23. When any member moves, creates, assigns, or comments on a task in a project, all other members currently viewing that project's board see the change live, no refresh. 24. When a user is assigned a task, their "Assigned to me" view updates live. 25. The socket must be authenticated, and a user must only receive events for projects they are a member of. Explain in the README how you secured the socket and how you scope events to the right people (not a global broadcast everyone hears). 26. Handle disconnects sensibly: if the socket drops, the app still works (refresh gets current state) and reconnects when possible. Describe your approach in the README.

**Dashboard** 27. Show for the logged-in user: number of projects they are in, tasks assigned to them by status, tasks they completed this week, the project with the most open tasks, and a personal recent-activity feed.

### Frontend quality requirements (not optional)

These apply across the whole app, we will check them:

- Every data-loading view shows a **loading state**, and a **clear error state** when a request fails (not a blank screen or a console error).
- Forms show **validation errors inline** (e.g., "Due date cannot be in the past"), not just an alert or a crash.
- Buttons that trigger requests are disabled / show progress while the request is in flight (no double-submit).

### Stretch goals (optional, only if core is rock solid)

Pick at most one or two.

- A few automated tests on critical paths (auth, role enforcement, assignment rules).
- Dockerized with `docker-compose` (frontend + backend + DB, one command).
- Drag-and-drop to move tasks between columns.
- Optimistic UI updates with rollback on failure.
- Deploy it live (Vercel / Render / Railway / Netlify free tiers).

## Tech you can use

The app must have a real backend and a real database.

- **Frontend:** Next.js / React / Vue.js
- **Backend:** Nest.js,  OR Python + FastAPI/Flask/Django.
- **Database:** MongoDB, PostgreSQL, or MySQL. (SQLite only if you justify it.)
- **Auth:** JWT with refresh tokens (required, see item 2).
- **Real-time:** WebSockets (required, see items 22 to 26).
- **Styling:** Anything usable and intentional.

## About using AI tools (read this honestly)

You may use ChatGPT, Claude, Copilot, or any AI tool. We use them too. We are not trying to catch you using AI.

The deal: **in the follow-up call we will pick parts of your code, ask you to explain them, make a small live change, and debug something.** This task has enough moving parts (refresh tokens, project membership, roles, assignment rules, real-time scoping) that copy-pasted code you do not understand will not survive questioning.

Honest advice:

- Use AI to learn and unblock, not to think for you.
- Read and understand everything before submitting. If AI wrote something you cannot explain, learn it or remove it.
- "I used AI here, here is what I learned and changed" scores positively.

## What to submit

### 1. GitHub repository (public)

- Run instructions that work on a clean clone.
- `.env.example` with required variables (no real secrets).
- A seed script creating at least two test users and one shared project with a few tasks (one already assigned across users), so we can test the collaboration and real-time flows immediately.
- Granular commit history. Commit as you build.

### 2. README.md

Include:

- What the app does (short paragraph).
- How to run it (step by step, clean-clone tested).
- **Your data model:** collections/tables, and how users, projects, memberships, tasks, and comments relate. Diagram or clear text.
- Stack choice and why.
- **How your refresh-token flow works** (what is stored where, what happens on expiry).
- **How your WebSocket setup works:** how you authenticate the socket, how you make sure events only reach members of the right project, and how you handle disconnects/reconnects.
- What was hard and how you worked through it.
- Known issues / what is incomplete.
- What you would improve with more time.
- Where you used AI and what you learned.

### 3. Demo video (4 to 6 minutes)

- Show, with two browser windows side by side: sign up two users, create a project, invite the second user, create and assign a task, move it across the board and watch it update live in the other window, comment on it, and the dashboard + activity feed.
- Talk through how it works. Casual is fine.